Get the stuff…

- Get the examples and slides:
  - git clone $USER@kestrel.hpc.nlr.gov:/nopt/nlr/apps/examples/gpu/h100
  - tar -xzf /nopt/nlr/apps/examples/gpu/h100.tgz
  - scp $USER@kestrel.hpc.nlr.gov:/nopt/nlr/apps/examples/gpu/h100.tgz
- Tarball with output from the runs
  - tar -xzf results.tgz
- Slides:
  - In the distribution

Kestrel GPUs
Building and Running

Timothy H. Kaiser, Ph.D.
tkaiser2@nlr.gov
August(ish) 2026

Kestrel Hardware Overview

Number of Nodes	Processors	Memory (DDR5)	Accelerators	Local Storage
2304	Dual socket Intel Xeon Sapphire Rapids 52-core processors (104 cores total)	2,240 x 256 GB 64 x 1 TB	N/A	256 x 1.92 TB NVMe M.2
156	Dual socket AMD Genoa 64-core processors (128 cores total)	108 x 384 GB 24 x 768 GB 24 x 1.5 TB	4 NVIDIA H100 SXM GPUs, 80 GB Memory	132 x 3.4 TB NVMe 24 x 14 TB NVMe
10	Dual socket Intel Xeon Sapphire Rapids 52-core processors (104 cores total)	10 x 2 TB	N/A	10 x 5.6 TB NVMe
8	Dual socket Intel Xeon Sapphire Rapids 52-core processors (104 cores total)	8 x 256 GB	2 NVIDIA A40 GPUs	2 x 3.84 TB NVMe

https://www.nlr.gov/hpc/kestrel-system-configuration.html

Kestrel Hardware Overview

Number of Nodes	Processors	Memory (DDR5)	Accelerators	Local Storage
2304	Dual socket Intel Xeon Sapphire Rapids 52-core processors (104 cores total)	2,240 x 256 GB 64 x 1 TB	N/A	256 x 1.92 TB NVMe M.2
156	Dual socket AMD Genoa 64-core processors (128 cores total)	108 x 384 GB 24 x 768 GB 24 x 1.5 TB	4 NVIDIA H100 SXM GPUs, 80 GB Memory	132 x 3.4 TB NVMe 24 x 14 TB NVMe
10	Dual socket Intel Xeon Sapphire Rapids 52-core processors (104 cores total)	10 x 2 TB	N/A	10 x 5.6 TB NVMe
8	Dual socket Intel Xeon Sapphire Rapids 52-core processors (104 cores total)	8 x 256 GB	2 NVIDIA A40 GPUs	2 x 3.84 TB NVMe

https://www.nlr.gov/hpc/kestrel-system-configuration.html

Kestrel GPU login nodes

- kestrel-gpu.hpc.nlr.gov
  - kl5.hpc.nlr.gov
  - kl6.hpc.nlr.gov
- Compiles for GPU nodes should be done on GPU login or compute nodes
- GPU jobs should be submitted from GPU login nodes
- Will GPU jobs run from or apps built on CPU login node work?
  - Maybe
  - Maybe not
  - For you own sanity, just don’t

NEW configuration for: gpu debug partition

- One node with 1 GPU visible

```
salloc --account=XXXX --nodes=1 --time=01:00:00 --partition=debug-gpu --gres=gpu:h100:1
```

- One node with 2 GPU visible

```
salloc --account=XXXX --nodes=1 --time=01:00:00 --partition=debug-gpu --gres=gpu:h100:2
```

- Two nodes with 1 GPU visible per node

```
salloc --account=XXXX --nodes=2 --time=01:00:00 --partition=debug-gpu -gres=gpu:h100:1
```

- Two nodes with 2 GPU visible per node

```
salloc --account=XXXX --nodes=2 --time=01:00:00 --partition=debug-gpu -gres=gpu:h100:2
```

Quick Start with just MPI

```
#!/bin/bash
#SBATCH --time=0:10:00
#SBATCH --partition=gpu-h100
#SBATCH --nodes=1
##### Don't need gpus for this run
##SBATCH --gres=gpu:h100:4
#SBATCH --exclusive
#SBATCH --output=quick.out
#SBATCH --error=quick.out

…
```

From kl5 or kl6

sbatch --account=#### quick

From other login nodes

sbatch --account=#### --partition=debug quick

```
[tkaiser2@kl5 h100]$cat quick | grep Compiling
echo Compiling with default compilers
echo Compiling and running with PrgEnv-cray
echo Compiling and running with PrgEnv-gnu
echo Compiling and running with PrgEnv-intel
echo Compiling and running with PrgEnv-nvidia
echo Compiling and running with IntelMPI
echo Compiling and running with OpenMPI with gcc
echo Compiling and running with OpenMPI with Intel backend
```

Quick start Output

- Output form one of the tests
  - Shows nodes and tasks
  - MPI version
  - Backend compiler

```
Hello from x3100c0s5b0n0 #            0  of            2
 MPI VERSION    : CRAY MPICH version 8.1.32.110 (ANL base 3.4a2)
MPI BUILD INFO : Thu Feb 06 22:53 2025 (git hash f9c5634)

 compiler: GCC version 13.3.1 20240611 (Red Hat 13.3.1-2)
 Hello from x3100c0s5b0n0 #            1  of            2
```

Covering Today:

- Build and run on Kestrel's GPU nodes using several programming paradigms.
- Multiple GPUs & multiple nodes.
  - Pure Cuda programs
  - Cuda aware MPI programs
  - MPI programs without Cuda,
  - MPI programs with Cuda
  - MPI programs with Openacc
  - Pure Openacc programs.
  - Library routines
- We'll build with
  - Cray's standard programming environment
  - NVIDIA's environment
  - Gcc
  - Intel MPI
- A repository will be given with programs, along with build and run scripts and a driver script.

- *Will not get to it today but we include slides that:*
  - *Discuss building and running the multi-gpu python Tensorflow/Horovod module.*

Run all of the examples

- tar -xzf /nopt/nlr/apps/examples/gpu/h100.tgz
- cd h100
- sbatch --account=XXXX do all
- Builds and Runs all of the examples in about 22 minutes
- Have added some things since "friendly users"
  - Multinode/GPU stream benchmark
  - More MPI gpu aware examples (used for testing new versions of MPI)
  - Select base gcc, test subset, other minor enhancements
  - Script "onnodes" to show what's happening
  - Standard function to reset modules
  - Works on RHEL9 nodes

Compiling for GPUs

- Currently the GPU environment is only available on GPU nodes
- You must compile your apps
  - On GPU login nodes
    - ssh kl5.hpc.nlr.gov
    - ssh kl5.hpc.nlr.gov
  - Via an interactive session in the gpu-h100 partition
  - Within your "run" script as we do today
- Interactive session:

```
salloc --exclusive --mem=0 --nodes=1 -t 02:00:00
--partition=gpu-h100 --gres=gpu:h100:4
--account=XXXX
```

- Batch script should have:

```
#SBATCH --partition=gpu-h100
#SBATCH --gres=gpu:h100:4
```

About gcc

- Almost all compiling/running on a linux system will at some point reference or in some way use some portion of the GNU (gcc/gfortran/linker) system
- Kestrel has many versions of gcc
- They fall in three categories
  - Native to the Operating system: 11.5
  - Supplement to the OS by Cray: 12.2, 13.2
    - ml cpe-stack gcc-native/12.2
    - ml cpe-stack gcc-native
  - Built by NLR: 14.2
    - ml gcc/14.2

More module Info

- The command module avail will show all modules
- module spider gcc will show modules pertaining to gcc
- Any module with nlr in the path is locally written
- Any module that stars with PrgEnv- give Cray MPI with someones compiler backend
  - Not all PrgEnv- modules work
  - Kept around to avoid breaking other things.

We want to start with a known Environment

- All our examples start with:

  module reset
- Starts fresh
- To get access to the Cray environments
  - ml cpe-stack
  - Gives us access to the latest **C**ray **P**rograming **E**nvironment modules => compilers, libs...
- To get access to the Intel environments
  - ml oneapi

After module reset

```
[tkaiser2@x3102c0s13b0n0 nvidiaopenmpi]$module reset
[tkaiser2@x3102c0s13b0n0 nvidiaopenmpi]$ml

Currently Loaded Modules:
  1) Core/26.05   2) application/26.05   3) DefApps
```

- To get access to the Cray environments
  - ml cpe-stack
  - Gives us access to the latest **C**ray **P**rograming **E**nvironment modules => compilers, libs...
- To get access to the Intel environments
  - ml oneapi

Our Examples

- Our driver script is just do all
- Each example directory contains a file "doit"
- Our driver looks for each directory with an example; Goes there and sources doit
- You can run a subset of the tests by setting the variable doits

Our Examples

```
[tkaiser2@kl1 h100]$find . -name doit
./cuda/gccalso/doit
./cuda/cray/doit
./cuda/nvidia/doit
./cudalib/factor/doit
./cudalib/fft/doit
./openacc/cray/doit
./openacc/nvidia/doit
./mpi/openacc/cray/doit
./mpi/openacc/nvidia/nvidiaopenmpi/doit
./mpi/cudaaware/doit
./mpi/normal/intel+abi/doit
./mpi/normal/cray/doit
./mpi/normal/nvidia/nvidiaopenmpi/doit
./mpi/withcuda/openmpi/doit
./mpi/withcuda/cray/doit
./mpi/withcuda/nvidia/nvidiaopenmpi/doit
[tkaiser2@kl1 h100]$
```

Name of directory says what we are testing

Driver Script

```
#!/bin/bash
#SBATCH --time=0:30:00
#SBATCH --partition=gpu-h100
#SBATCH --nodes=2
#SBATCH --gres=gpu:h100:4
#SBATCH --exclusive
#SBATCH --output=output-%j.out
#SBATCH --error=infor-%j.out

func () { typeset -f $1 || alias $1; }
func module > module.$SLURM_JOB_ID

alias srun="/nopt/slurm/current/bin/srun --time=00:05:00 $@"
if echo $SLURM_SUBMIT_HOST | egrep "kl5|kl6" >> /dev/null  ; then : ; else echo Run script from a GPU node; exit ; fi

# a simple timer
dt ()
{
    now=`date +"%s.%N"`;
    if (( $# > 0 )); then
        rtn=$(printf "%0.3f" `echo $now - $1 | bc`);
    else
        rtn=$(printf "%0.3f" `echo $now`);
    fi;
    echo $rtn
}

printenv > env-$SLURM_JOB_ID.out
cat $0 > script-$SLURM_JOB_ID.out
```

Driver Script

```
#runs script to restore our environment
module reset

#some possible values for gcc module
#export MYGCC=gcc-stdalone/13.1.0
#export MYGCC=gcc/14.2.0

if [ -z ${MYGCC+x} ]; then export MYGCC=gcc/14.2.0  ; else echo MYGCC already set ; fi
echo MYGCC=$MYGCC

if [ -z ${doits+x} ]; then
     doits=`find . -name doit | sort -t/ -k2,2`
else
     echo doits already set
fi

for x in $doits ; do
     echo running example in `dirname $x`
done
```

Driver Script

```
startdir=`pwd`
t1=`dt`
export MYSAVEP=$PATH
export MYSAVEL=$LD_LIBRARY_PATH
for x in $doits ; do
 dir=`dirname $x`
 echo ++++++++ $dir >&2
 echo ++++++++ $dir
 cd $dir
 tbegin=`dt`
 . doit | tee  $SLURM_JOB_ID
 echo Runtime `dt $tbegin` $dir `dt $t1` total
 cd $startdir
 echo resting environment
 module reset
 export PATH=$MYSAVEP
 export LD_LIBRARY_PATH=$MYSAVEL
 ml 2>&1
 echo PATH $PATH
 echo LD_LIBRARY_PATH $LD_LIBRARY_PATH
done
echo FINISHED `dt $t1`
echo FINISHED `dt $t1` >&2

# post  (this is optional)
mkdir -p /scratch/$USER/gputest/$SLURM_JOB_ID
cp *out  /scratch/$USER/gputest/$SLURM_JOB_ID
# . cleanup
```

More notes

- While all of these examples work...
  - In most cases there maybe other combinations of modules that can be used
  - These might not be the best
  - The machine is evolving
    - Somethings might not work later
    - There are already know changes coming to Kestrel that will deprecate some of the module choices

Common issues

- Can’t find library at run time
  - Need to set LD_LIBRARY_PATH to point to directory containing the library. Try to load modules at run time.
- Module xxx is not compatible with your cray-libsci
  - Load an different version: cray-libsci/22.10.1.2 or cray-libsci/22.12.1.1 or cray-libsci/23.05.1.4
- Can’t find some function in the c++ library
  - Load a newer version of gcc
- At link time libgcc_s.so.1: file not recognized: File format not recognized
  - Linker is missing after some combinations of loads. module load binutils
- Examples shown here don’t work
  - Make sure you are running and or launching from a GPU node
- cc1: error: bad value ‘znver4’ for ‘-march=’ switch
  - -march=skylake
    - Or
  - module load craype-x86-milan
- Package 'nccl', required by 'virtual:world', not found…
  - module unload nvhpc

Let's get into it

- We'll look at each example directory
- Discuss compilers and setting are being used
- Discuss the example programs
- Show portions of the "doit" script that are more important
- NOTE: there are more detailed explanations in the details.md file

./cuda/cray

Working directory cuda/cray

Steam.cu runs a standard benchmark showing the computational speed of the gpu for simple math operations. "srun" is not required here but we use it to run on both nodes.

Key Parts

```
ml cpe-stack/25.03
ml PrgEnv-nvidia/8.6.0

nvcc -std=c++11 -ccbin=g++ stream.cu -arch=sm_90 -o stream.sm_90


# Run on all of our nodes
nlist=`scontrol show hostnames | sort -u`
for l in $nlist ; do
  echo $l
  for GPU in 0 1 2 3 ; do
# stream.cu will read the GPU on which to run from the command line
      srun -n 1 --nodes=1 -w $l ./stream.sm_90 -g $GPU
  done
  echo
done
```

./cuda/gccalso

Working directory cudalib/gccalso

Key Parts

Here we break the compile into two parts, normal C and Cuda. "srun" is not required here but we use it to run on both nodes.

```
ml nvhpc


g++ -c normal.c
nvcc -std=c++11 -arch=sm_90 cuda.cu normal.o -o stream.sm_90


# Run on all of our nodes
nlist=`scontrol show hostnames | sort -u`
for l in $nlist ; do
  echo $l
  for GPU in 0 1 2 3 ; do
# stream.cu will read the GPU on which to run from the command line
      srun -n 1 --nodes=1 -w $l ./stream.sm_90 -g $GPU
  done
  echo
done
```

Simple MPI - Hello World in Fortran and C

Working directory mpi/normal/cray

Cray MPI with Cray compilers

For PrgEnv-cray we can build with cc and ftn which are wrappers for MPI compilers.
We use srun to run on 1 and two nodes with 4 tasks total.

Key Parts

```
ml cpe-stack/25.03
ml PrgEnv-cray
ml craype-x86-genoa

cc helloc.c -o helloc
ftn hellof.f90 -o hellof

for arg in "--tasks-per-node=2 -n 4 --nodes=2" "-n 2 --nodes=1 --tasks-per-node=2" ; do
   echo running Fortran version
   srun $arg hellof
   echo
   echo running C version
   srun $arg helloc
   echo
done
```

Simple MPI - Hello World in Fortran and C

Working directory mpi/normal/cray

Key Parts

Cray MPI with Intel Compilers

For PrgEnv-cray we can build with cc and ftn which are wrappers for MPI compilers.
We use srun to run on 1 and two nodes with 4 tasks total.

```
ml cpe-stack/25.03
ml PrgEnv-intel
ml craype-x86-genoa

cc helloc.c -o helloc
ftn hellof.f90 -o hellof

for arg in "--tasks-per-node=2 -n 4 --nodes=2" "-n 2 --nodes=1 --tasks-per-node=2" ; do
   echo running Fortran version
   srun $arg hellof
   echo
   echo running C version
   srun $arg helloc
   echo
done
```

Simple MPI - Hello World in Fortran and C

Working directory mpi/normal/cray

Cray MPI with gnu Compilers

Key Parts

For PrgEnv-cray we can build with cc and ftn which are wrappers for MPI compilers.
We need the march flag because the default gnu compilers cannot created instructions for Genoa cpus.

```
ml cpe-stack/25.03
ml PrgEnv-gnu/8.6.0
ml craype-x86-genoa

cc  -march=znver3 helloc.c -o helloc
ftn -march=znver3 hellof.f90 -o hellof

for arg in "--tasks-per-node=2 -n 4 --nodes=2" "-n 2 --nodes=1 --tasks-per-node=2" ; do
   echo running Fortran version
   srun $arg hellof
   echo
   echo running C version
   srun $arg helloc
   echo
done
```

Simple MPI - Hello World in Fortran and C

Working directory mpi/normal/nvidia

Key Parts #1

Nvidia nvhpc compilers

The Nvidia HPC suite ships with MPI but it is not part of the module. Here we "find it". We could just hardwire the path but this shows one way that we might do it in a script.

```
ml  nvhpc/26.1
#MPI is not in the module. We add it here.
export MYMPIDIR=`dirname $(dirname $(dirname $(which nvcc)))`
export MYMPIDIR=$MYMPIDIR/comm_libs/hpcx/bin
export PATH=$MYMPIDIR:$PATH
unset MYMPIDIR

mpicc helloc.c -o helloc
mpif90 hellof.f90 -o hellof
```

Simple MPI - Hello World in Fortran and C

Working directory mpi/normal/cray

Nvidia nvhpc compilers

We use mpirun instead of srun. The flag

```
--mca coll_hcoll_enable
```

suppresses a warning about the network configuration

Key Parts #2

```
for arg in "-N 2" "-n 2" ; do
   echo running Fortran version
   mpirun $arg --mca coll_hcoll_enable 0 hellof
   echo
   echo running C version
   mpirun $arg --mca coll_hcoll_enable 0 helloc
   echo
done
```

Simple MPI - Hello World in Fortran and C

Working directory mpi/normal/intel+abi

Key Parts #1

```
ml cpe-stack/25.03
ml libfabric/1.22.0
ml oneapi
ml intel-oneapi-mpi

mpiicx helloc.c -o helloc
mpifc hellof.f90 -o hellof

srun -n 4 hellof
srun -n 4 hello
```

Intel MPI and Intel MPI with Cray overload (abi)

We start with compiling and running with Intel MPI and we use Intel's miiicx and mpifc to get Intel backend compilers

Overloading Intel MPI with Cray MPI

- You can replace Intel MPI with Cray MPI at run time
- This should be possible with a simple module load
- It's broken
- Hack:
  - Build with Intel as shown in the previous slide
  - Load the cray software stack
    - ml cpe-stack/25.03
    - ml intel/2025.3 craype/2.7.34
    - ml cray-mpich-abi/8.1.32

Overloading Intel MPI with Cray MPI

- Hack continued:
  - # grab the environmental variable:
  - export MYLDPATH=$CRAY_LD_LIBRARY_PATH
  - # reload Intel environment

```
module reset
echo $MYLDPATH
ml cpe-stack/25.03
ml libfabric/1.22.0
ml oneapi
ml intel-oneapi-mpi
```

  - export LD_LIBRARY_PATH=$MYLDPATH:$LD_LIBRARY_PATH
  - Use srun to run you app and it will call Cray MPI

Back to GPUs

- OpenACC
- MPI with OpenACC
- MPI with cuda
  - Send data to/from GPU via "normal" push/pull and MPI calls
  - MPI coda that calls a cuda kernel
- Cuda aware MPI - MPI
  - calls send/recv data directly to/from GPUs
- Finish up with some libraries.

Openacc

- Our example is from NVIDIA
- We run on each of the GPUs one at a time by setting CUDA_VISABLE_DEVICES.
- We launch with srun or mpirun although it is not needed in this case.
- Use:
  - PrgEnv-nvidia/8.6.0
  - nvhpc/26.1

Openacc

Working directories
openacc/nvidia
openacc/cray

Key Parts
nvhpc

```
ml nvhpc/26.1
nvc -fast -Minline -Minfo \
  -acc -DFP64 nbodyacc2.c -o nbody
```

Key Parts
PrgEnv-nvidia/8.6.0

```
ml cpe-stack
ml  PrgEnv-nvidia/8.6.0
nvc -fast -Minline -Minfo \
  -acc -DFP64 nbodyacc2.c -o nbody
```

```
# Run on all of our nodes
nlist=`scontrol show hostnames | sort -u`
for l in $nlist ; do
  echo $l
  for GPU in 0 1 2 3 ; do
# This is one way to set the GPU on which a openacc program runs.
      export CUDA_VISIBLE_DEVICES=$GPU
      echo running on gpu $CUDA_VISIBLE_DEVICES
      srun -n 1 --nodes=1 -w $l ./nbody
  done
  echo
done
```

MPI and Openacc

MPI and Openacc using PrgEnv-nvhpc

Working directory mpi/openacc/cray

- This example just combines some Openacc Code with MPI calls.
- It is a contrived example. It just uses MPI to ask each node to do the same calculation with Openacc and then uses MPI to gather and report stats.
- We run two cases using the module PrgEnv-nvidia and nvhpc
- The flags -acc -Minfo -fast enable openacc, source build information and optimization
- PrgEnv-nvidia works as expected
- The nvhpc module does not bring in MPI, we have to find it and we need to use mpirun instead of srun

MPI and Openacc using PrgEnv-nvhpc

```
# Start from a known module state, the default
module reset

# Load modules
if [ -z ${MYGCC+x} ]; then module load gcc ; else module load $MYGCC ; fi
ml cpe-stack
ml PrgEnv-nvidia
ml craype-x86-genoa

<< ++++
 Compile our program.

 Here we use cc and ftn.  These are wrappers
 that point to Cray C (clang) Cray Fortran
 and Cray MPI. cc and ftn are part of PrgEnv-cray
 which is part of the default setup.
++++

cc -acc -Minline -Minfo   -fast acc_c3.c  -o jacobi

# We run with 4 tasks per nodes.
srun  --tasks-per-node=4 --nodes=2 -n 8 ./jacobi 46000 46000 5 nvidia
```

Working directory openacc/nvidia/nvidiaopenmpi

MPI and Openacc using nvhpc

```
# Start from a known module state, the default
module reset
# Load modules
if [ -z ${MYGCC+x} ]; then module load gcc ; else module load $MYGCC ; fi
ml nvhpc

# the module does not have the path for mpi we need to find it

export MYDIR=$(dirname $(dirname $(dirname `which nvcc`)))
find $MYDIR -name mpicc | sort | tail -1
export MYMPIDIR=$(dirname $(find $MYDIR -name mpicc | sort | tail -1))
export PATH=$MYMPIDIR:$PATH

<< ++++
 Compile our program Here we use mpicc and mpif90.  There is support for Cuda
 but we are not using it in this case but we are using openacc.
++++

mpicc -acc -Minfo  -fast acc_c3.c -o jacobi

# We run with 4 tasks per nodes.
# This version of MPI does not support srun so we use mpirun
mpirun -mca coll_hcoll_enable 0 -N 4 ./jacobi 46000 46000 5 nvidia
```

MPI and Cuda

MPI and Cuda

- We'll run a MPI benchmark with data on a GPU.
- We'll first look at the case where we copy it to the CPU, use MPI it to copy to another task and finally move it to the GPU. We'll also look at direct gpu to gpu transfers.
- The second program in our test runs the Stream benchmark using MPI to run it on each GPU in parallel.

MPI and cuda using PrgEnv-nvidia

Working directory mpi/withcuda/cray

Cray MPI with cuda

We compile our MI benchmark with CC, the Cray MPI wrapper
Then we run it with two tasks on a node and 1 task on each of two nodes.

Key Parts #1

```
ml cpe-stack
ml PrgEnv-nvidia
ml craype-x86-genoa
ml cudatoolkit
ml craype-accel-nvidia90

CC -gpu=cc90   ping_pong_cuda_staged.cu -o staged

srun  --nodes=1 --tasks-per-node=2 ./staged
srun  --nodes=2 --tasks-per-node=1 ./staged
```

MPI and cuda using PrgEnv-nvidia

Cray MPI with cuda

This is a parallel version of stream.cu. It runs stream in parallel on all gpus with MPI as the manager. CC is broken for programs containing some (most0 cuda calls. So here we need to manually build with nvcc and pull in MPI. We run it with 4 tasks/node.

Key parts #2

```
#CC -gpu=cc90  -DNTIMES=1000  mstream.cu -o mstream
export MYMPIDIR=$(dirname $(dirname $(which mpicc)))
echo $MYMPIDIR
nvcc -L$MYMPIDIR/lib -I$MYMPIDIR/include -arch=native  -lmpi -DNTIMES=1000  mstream.cu -o mstream
export LD_LIBRARY_PATH=$MYMPIDIR/lib:$LD_LIBRARY_PATH


srun --tasks-per-node=4 --nodes=2 -n 8 ./mstream -n $VSIZE
```

MPI and cuda using nvhpc

Working directory mpi/withcuda/nvhpc

nvhpc with cuda

As we have seen in an earlier example MPI does not get pulled in with the nvhpc module. We need to find it as shown here. We can then compile with mpiCC.

Key Parts #1

```
ml  nvhpc/26.1
#MPI is not in the module. We add it here.
export MYMPIDIR=`dirname $(dirname $(dirname $(which nvcc)))`
MPIH=$(dirname $(find $MYMPIDIR  -name mpi.h | sort | tail -1))
MPIL=$(dirname $(find $MYMPIDIR  -name libmpi.so | sort | tail -1))
MPIB=$(echo $MPIH | sed s/include/bin/)
echo $MPIH
echo $MPIL
echo $MPIB
export PATH=$MPIB:$PATH

mpiCC ping_pong_cuda_staged.cu -o staged
```

MPI and cuda using nvhpc

nvhpc with cuda

We run with mpirun using the flag shown below to suppress warnings about the network. We also compile and run the MPI version of stream.

Key parts #2

```
echo Run on a single node
mpirun  --mca coll ^hcoll  -n 2 -N 2 ./staged

echo Run on two nodes
# --mca coll ^hcoll turn off a warning about not having InfiniBand
mpirun  --mca coll ^hcoll -n 2 -N 1 ./staged

nvcc -I$MPIH -L$MPIL -lmpi -arch=native  -DNTIMES=1000  mstream.cu -o mstream
mpirun  --mca coll ^hcoll -n 8 -N 4  ./mstream -n $VSIZE
```

MPI and cuda using nvhpc

nvhpc with cuda

We run with mpirun using the flag shown below to suppress warnings about the network. We also compile and run the MPI version of stream.

Key parts #2

```
echo Run on a single node
mpirun  --mca coll ^hcoll  -n 2 -N 2 ./staged

echo Run on two nodes
# --mca coll ^hcoll turn off a warning about not having InfiniBand
mpirun  --mca coll ^hcoll -n 2 -N 1 ./staged

nvcc -I$MPIH -L$MPIL -lmpi -arch=native  -DNTIMES=1000  mstream.cu -o mstream
mpirun  --mca coll ^hcoll -n 8 -N 4  ./mstream -n $VSIZE
```

MPI and cuda using NLR openmpi

Working directory mpi/withcuda/openmpi

nvhpc with cuda

Here we pull in cuda and openmpi. This version of MPI was built with gcc not Nvidia's compilers. The mpiCC command does not work with nvcc directly because of some incompatibilities of flags from gcc. So we compile with nvcc and point to the MPI libraries and run with mpirun

Key Parts #1

```
ml gcc/14.2.0
ml cuda/13.2
ml openmpi/5.0.3

MYMPIPATH=$(dirname $(dirname $(which mpicc)))
IPATH=$MYMPIPATH/include
LPATH=$MYMPIPATH/lib

nvcc -L$LPATH -I$IPATH -lmpi -arch=native ping_pong_cuda_staged.cu -o staged

IRFLAG="-mca mtl ^ofi --mca btl ^ofi"
mpirun $IRFLAG -n 2  ./staged
mpirun $IRFLAG -n 2 -N 1  ./staged
```

MPI and cuda using NLR openmpi

nvhpc with cuda

The parallel version of stream is again compiled with mvcc and run with mpirun

Key Parts #2

```
IRFLAG="-mca mtl ^ofi --mca btl ^ofi"
echo running multi-gpu stream
nvcc -L$LPATH -I$IPATH -lmpi  -arch=native  -lmpi -DNTIMES=1000  mstream.cu -o mstream

export VSIZE=3300000000
export VSIZE=330000000
mpirun $IRFLAG -N 4  ./mstream -n $VSIZE
```

Cuda Aware MPI

We'll again run a MPI benchmark with data on a GPU.

In this second case the data goes directly from GPU to GPU without going to the CPU. This is in general much faster. (The NLR version of MPI is not very fast. This is under investigation.)

The difference in source is shown on the next slide. Note in the second case we don't have the cudaMemcopy calls.

Our working directory for this is example is

`mpi/cudaaware`

We have a single script that builds/runs with:

- PrgEnv-nvidia & cuda
- OpenMPI & cuda
- Nvhpc

The important parts of each case are given on slides below.

Cuda MPI Staged vs Cuda Aware (Direct)

Staged

```
for(int i=1; i<=loop_count; i++){
	if(rank == 0){
		cudaMemcpy(A, d_A, N*sizeof(double), cudaMemcpyDeviceToHost) ;
		MPI_Send(A, N, MPI_DOUBLE, 1, tag1, MPI_COMM_WORLD);
		MPI_Recv(A, N, MPI_DOUBLE, 1, tag2, MPI_COMM_WORLD, &stat);
		cudaMemcpy(d_A, A, N*sizeof(double), cudaMemcpyHostToDevice) ;
	}
	else if(rank == 1){
		MPI_Recv(A, N, MPI_DOUBLE, 0, tag1, MPI_COMM_WORLD, &stat);
		cudaMemcpy(d_A, A, N*sizeof(double), cudaMemcpyHostToDevice) ;
		cudaMemcpy(A, d_A, N*sizeof(double), cudaMemcpyDeviceToHost) ;
		MPI_Send(A, N, MPI_DOUBLE, 0, tag2, MPI_COMM_WORLD);
	}
}
```

Cuda Aware

```
for(int i=1; i<=loop_count; i++){
	if(rank == 0){
		MPI_Send(d_A, N, MPI_DOUBLE, 1, tag1, MPI_COMM_WORLD);
		MPI_Recv(d_A, N, MPI_DOUBLE, 1, tag2, MPI_COMM_WORLD, &stat);
	}
	else if(rank == 1){
		MPI_Recv(d_A, N, MPI_DOUBLE, 0, tag1, MPI_COMM_WORLD, &stat);
		MPI_Send(d_A, N, MPI_DOUBLE, 0, tag2, MPI_COMM_WORLD);
	}
}
```

Cuda Aware MPI - PrgEnv-nvidia

Working directorympi/cudaaware

```
ml cpe-stack
ml PrgEnv-nvidia
ml craype-x86-genoa
ml cudatoolkit
ml craype-accel-nvidia90

<< ++++
 Compile our program.

 Here we use cc and CC.  These are wrappers
 that point to Cray MPI but use Nvidia backend
 comilers.
++++

CC  -target-accel=nvidia90 -lcudart ping_pong_cuda_aware.cu -o pp_cuda_aware

export MPICH_GPU_SUPPORT_ENABLED=1
export MPICH_OFI_NIC_POLICY=GPU
srun -n 2 --nodes=1 ./pp_cuda_aware
srun --tasks-per-node=1 --nodes=2 ./pp_cuda_aware
unset MPICH_GPU_SUPPORT_ENABLED
unset MPICH_OFI_NIC_POLICY
```

We must set:

```
export MPICH_GPU_SUPPORT_ENABLED=1
export MPICH_OFI_NIC_POLICY=GPU
```

Cuda Aware MPI - OpenMPI and cuda

Working directorympi/cudaaware

```
ml gcc/14.2.0
ml cuda/13.2
ml openmpi/5.0.3

<< ++++
 Compile our program.

 The mpi wrappers don't work for nvidia compilers.
 There are some flags they do not understand. So we
 call nvcc with the required command line options.

Here we find the include and lib paths.
++++

MYMPIPATH=$(dirname $(dirname $(which mpicc)))
IPATH=$MYMPIPATH/include
LPATH=$MYMPIPATH/lib

nvcc -L$LPATH -I$IPATH -lmpi -arch=native  ping_pong_cuda_aware.cu -o pp_cuda_aware_openmpi

IRFLAG="-mca mtl ^ofi --mca btl ^ofi"
mpirun $IRFLAG -n 2 ./pp_cuda_aware_openmpi
mpirun $IRFLAG -N 1 ./pp_cuda_aware_openmpi
```

We must set:

```
export MPICH_GPU_SUPPORT_ENABLED=1
export MPICH_OFI_NIC_POLICY=GPU
```

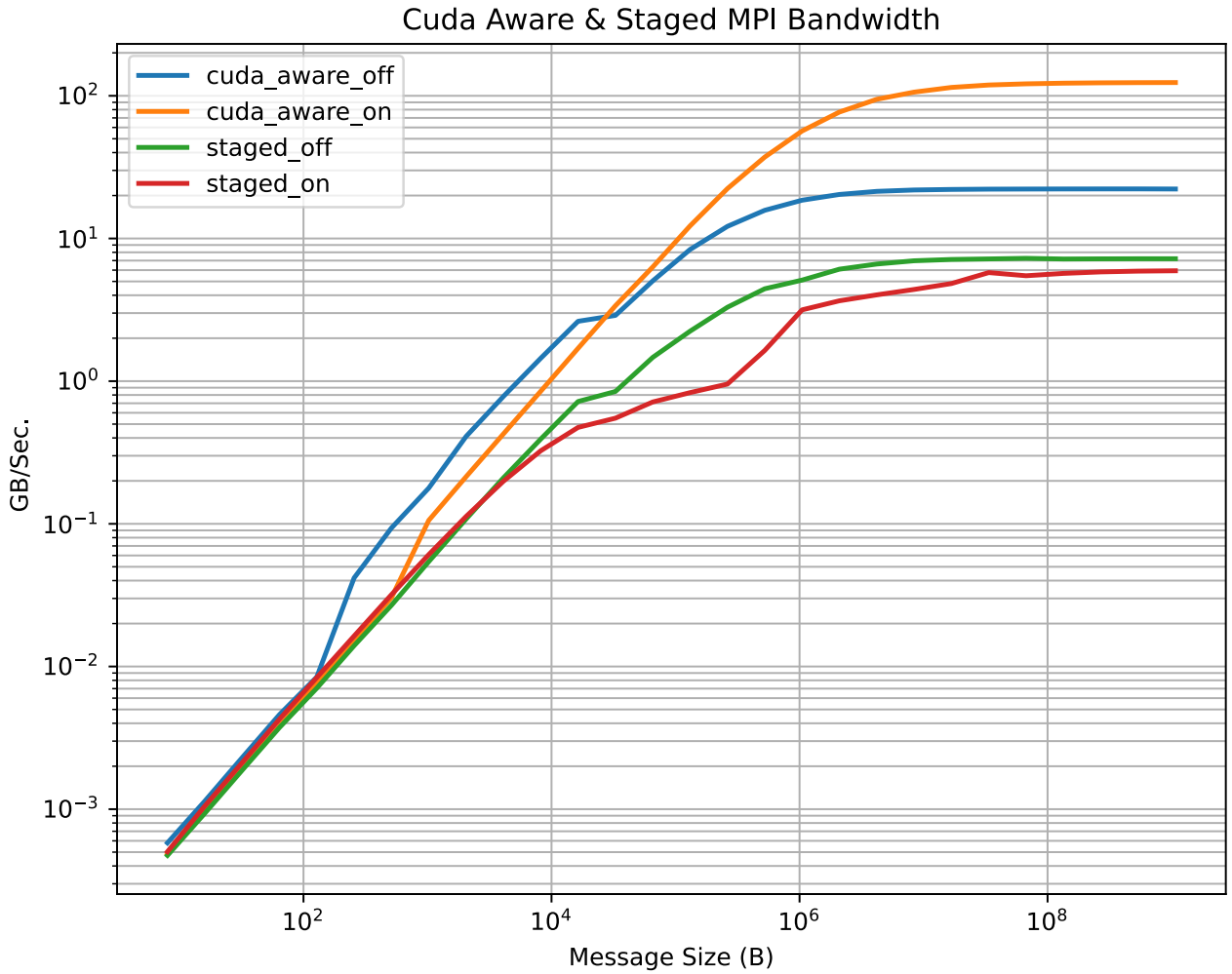
Cuda Aware MPI - nvhpc

Working directorympi/cudaaware

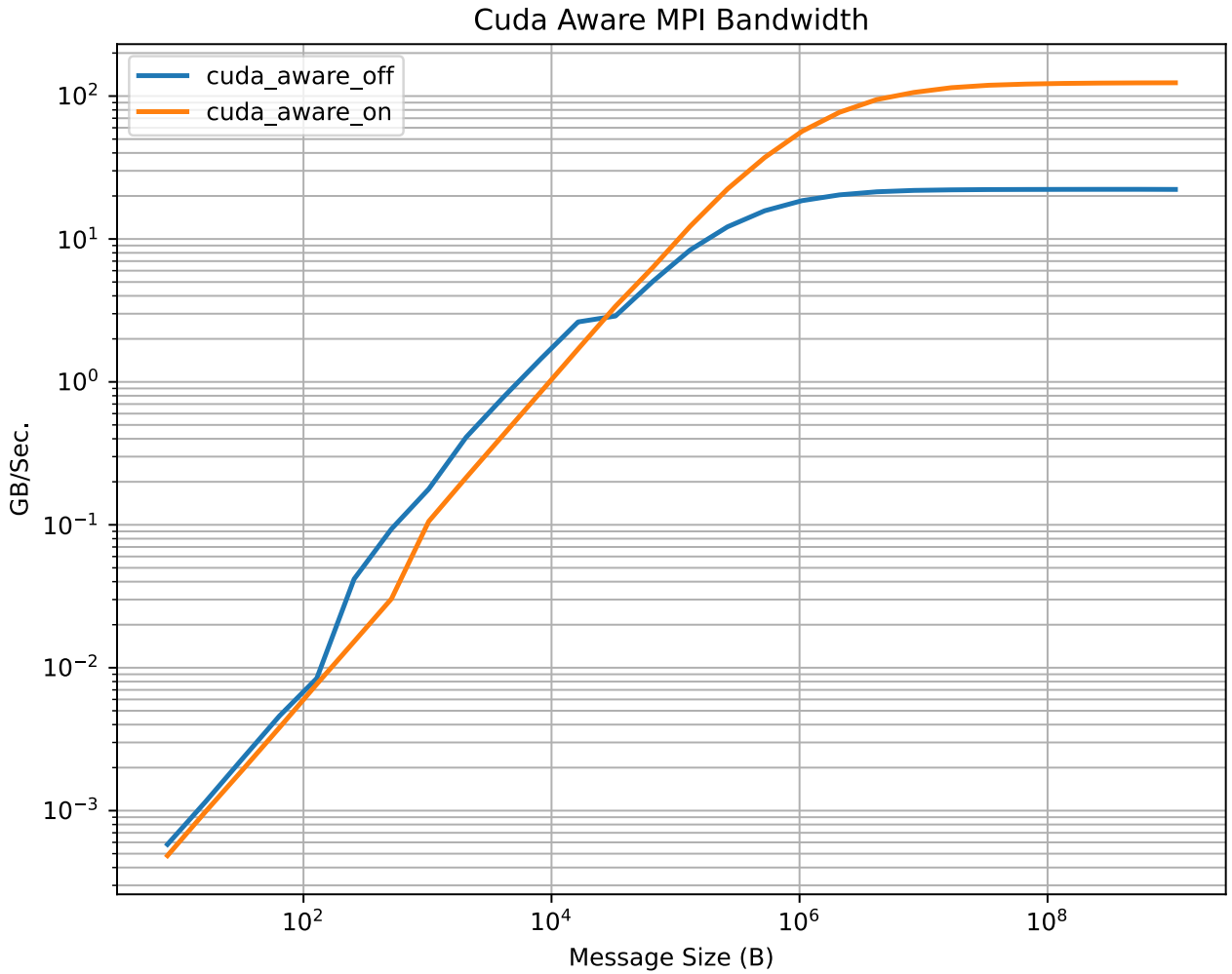
```
ml nvhpc

<< ++++

here we compile with nvcc and nvidia's MPI.  However,
again their mpicc does not work corrrectly.  So we
find nvcc, the lib, include,and bin .directories.  We
compile with nvcc and their lib and include directories
and run with their mpirun
++++

NVHPCDIR=$(dirname $(dirname $(dirname $(which nvcc))))
MPIH=$(dirname $(find $NVHPCDIR -name mpi.h | head -1))
MYMPIDIR=$(dirname $MPIH)
IPATH=$MYMPIDIR/include
LPATH=$MYMPIDIR/lib
BPATH=$MYMPIDIR/bin

nvcc -L$LPATH -I$IPATH -lmpi -arch=native ping_pong_cuda_aware.cu -o pp_nvhpc

echo mpirun $BPATH/mpirun
RTFLAG="--mca coll ^hcoll"
$BPATH/mpirun $RTFLAG -n 2 pp_nvhpc
$BPATH/mpirun $RTFLAG -N 1 pp_nvhpc
```

Again we must use nvcc and point to mpi

Cuda MPI Staged vs Cuda Aware (Direct)



Cuda MPI Staged vs Cuda Aware (Direct)



Libraries

- Writing cuda functions is relatively easy
- Getting cuda functions to work well is very difficult
- Use libraries when you can
  - NVIDIA has several
  - Many third party libs
  - Here we will look at NVIDIA libs and compare to Cray and Intel CPU libs

Linear Solve

Working directory cudalib/factor

We are going to do some linear solves. cusolver_getrf_example.cu is a slight modification of one of NVIDIA's examples. cpu.C uses lapack routines.

We'll build cpu.C against Cray's (libsci) and Intel's (MKL) libraries. The NVIDIA example uses NVIDIA's tools.

Here we just just build the the libsci version using one of the PrgEnv compilers

We build the GPU version with nvhpc.

Linear Solve

Building and running the CPU inversion program with Cray's library and Intel's MKL library

```
export OMP_NUM_THREADS=32

ml cpe-stack
ml PrgEnv-gnu

# Here we build the CPU version with libsci.
CC  -DMINE=$MSIZE  -fopenmp -march=native cpu.C -o invert.libsci

./invert.libsci

# We are going to compile the Intel version using
# the CPU environment
ml oneapi
ml intel-oneapi-mkl
icpx  -DMINE=$MSIZE -qopenmp -D__INTEL__ -march=native cpu.C -mkl -lmkl_rt -o invert.mkl


./invert.mkl
```

Linear Solve

Building and running the GPU inversion program with nvhpc

```
<< ++++
 Compile our GPU programs.
 The module nvhpc gives us access to Nvidia's compilers
 nvc, nvc++, nvcc, nvfortran as well as the Portland Group
 compilers which are actually links to these.

 We need to find libcusolver.so  and libnvJitLink.so.
++++
ml cpe-stack
ml nvhpc
ml craype-x86-genoa
# GPU version with libcusolver
export MYMPIDIR=`dirname $(dirname $(dirname $(which nvcc)))`
L1=`find $MYMPIDIR -name libcusolver.so | sort | head -1`
L1=`dirname $L1`
L2=`find $MYMPIDIR -name libnvJitLink.so | sort | head -1`
L2=`dirname $L2`


nvcc  -DMINE=$MSIZE -L$L1 -lcusolver -L$L2 -lnvJitLink cusolver_getrf_example.cu -o invert.gpu

for GPU in 0 1 2 3 ; do
  ./invert.gpu $GPU
done
```

FFT

Working directory cudalib/fft

- Here we are doing a fft on a cube.
- The GPU version will work on multiple GPUs.
- For the CPU version we call Cray's libsci version of fftw.
- We're using on of the NVIDIA's toolsets for the GPU version

FFT

Building our GPU version.
We need to find our libraries.

Key Parts #1

```
ml nvhpc

<< ++++
 Compile our GPU programs.
 The module nvhpc-stdalone gives us access to Nvidia's compilers
 nvc, nvc++, nvcc, nvfortran as well as the Portland Group
 compilers which are actually links to these.
++++

export MYMPIDIR=$(dirname $(dirname $(dirname  $dirname `which nvcc`)))
export L1=$(dirname $(find $MYMPIDIR  -name "libcufft.so*" | sort | head -1))

nvcc -O3 -forward-unknown-to-host-compiler  --generate-code=arch=compute_90,code=[compute_90,sm_90] \
         -std=c++11 -x cu 3d_mgpu_c2c_example.cpp -c
nvcc  -o 3dfft 3d_mgpu_c2c_example.o -L$L1 -lcufft
```

FFT

Running our GPU version. Run our GPU version on a single and 4 GPUs. We do this twice in opposite orders.

Key Parts #2

```
: Run our program on a cube. The first parameter gives our cube size.
: 2048 should work on the H100s.
: Second parameter determines which algorithm runs first 1 GPU version or 4 GPU version
echo
echo
for DOIT in `seq 1 4` ; do
  echo set $DOIT
  echo ++++++++++++++
  echo RUN SINGLE GPU VERSION FIRST
  ./3dfft 512 1
  echo
  echo
  echo ++++++++++++++
  echo RUN FOUR GPU VERSION FIRST
  ./3dfft 512 2
  echo
  echo
done
```

FFT

Key Parts #3

FFTW from Craylibsci

```
# Build and run a fftw version
module reset
ml cpe-stack/25.03
ml PrgEnv-gnu/8.6.0
ml craype-x86-genoa
ml cray-fftw

cc -O3 fftw3d.c -o fftw3.exe

echo
echo
echo +++++++++++++++
echo run fftw libsci version
./fftw3.exe 512
```

FFT

For the CPU version we call Cray's libsci version of fftw.

```
: Build and run a fftw version
module restore
source /nopt/nlr/apps/gpu_stack/env_cpe23.sh
ml cray-fftw
cc -O3 fftw3d.c -o fftw3.exe


echo
echo
echo ++++++++++++++
echo run fftw libsic version
./fftw3.exe 512
```

Summary

- Showed how to build and run many types of GPU enabled applications on Kestrel
- Provided a repo and tar ball for the examples
  - git clone $USER@kestrel.hpc.nlr.gov:/nopt/nlr/apps/examples/gpu/h100
  - tar -xzf /nopt/nlr/apps/examples/gpu/h100.tgz
  - scp $USER@kestrel.hpc.nlr.gov:/nopt/nlr/apps/examples/gpu/h100.tgz
- Things will change. These scripts might not work in after updates
- Future work
  - Keep the scripts current
  - Add other options / improve Cuda aware MPI